



筑波大学

University of Tsukuba

Data Engineering I

Quiz 7

Mamanchuk Mykola, SID.202420671

November 28, 2024

Question (i): Modularity Q

Given an undirected graph with edges:

$$E = \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 5\}, \{3, 7\}, \{4, 5\}, \{4, 6\}, \{5, 6\}, \{5, 7\}, \{7, 8\}, \{7, 9\}, \{8, 9\}\}$$

We partition the graph into two clusters:

$$C_1 = \{1, 2, 3, 4, 5, 6\}$$

$$C_2 = \{7, 8, 9\}$$

The modularity Q is defined as:

$$Q = \frac{1}{2m} \sum_{u,v} \left[A_{uv} - \frac{k_u k_v}{2m} \right] \delta(c_u, c_v)$$

where:

- m is the total number of edges.
- A_{uv} is 1 if there is an edge between nodes u and v , 0 otherwise.
- k_u and k_v are the degrees of nodes u and v , respectively.
- $\delta(c_u, c_v)$ is 1 if nodes u and v are in the same cluster, 0 otherwise.

Answer: The modularity for the given partition is:

$$Q = 0.285$$

Question (ii): SCAN Algorithm Clustering

Applying the SCAN algorithm to the same graph with parameters:

- $\varepsilon = 0.8$
- $\mu = 2$

The structural similarity between two nodes u and v is calculated as:

$$s(u, v) = \frac{|N(u) \cap N(v)|}{\sqrt{|N(u)| \times |N(v)|}}$$

where $N(u)$ is the set of neighbors of u (including u itself).

A node is considered a **core node** if it has at least $\mu - 1$ neighbors in its ε -neighborhood (nodes with structural similarity $\geq \varepsilon$).

Answer: The SCAN algorithm identifies:

- **Clusters:**
 1. Cluster 1: {1, 2, 3}
 2. Cluster 2: {4, 6}
 3. Cluster 3: {8, 9}
- **Hubs:** {5, 7}
- **Outliers:** None

Listing assignment.py

```
1 import networkx as nx
2
3 #####
4 ### ASSIGNMENT 1 ###
5 #####
6
7 # Define the graph edges
8 edges = [
9     (1, 2), (1, 3), (2, 3),
10    (3, 5), (3, 7), (4, 5),
11    (4, 6), (5, 6), (5, 7),
12    (7, 8), (7, 9), (8, 9)
13 ]
14
15 # Create the graph
16 G = nx.Graph()
17 G.add_edges_from(edges)
18
19 # Define the clusters
20 C1 = {1, 2, 3, 4, 5, 6}
21 C2 = {7, 8, 9}
22 clusters = [C1, C2]
23
24 # Calculate Modularity Q
25 def calculate_modularity(G, clusters):
26     m = G.number_of_edges()
27     Q = 0.0
28     cluster_labels = {}
29     for idx, cluster in enumerate(clusters):
30         for node in cluster:
31             cluster_labels[node] = idx
32     for u, v in G.edges():
33         delta = 1 if cluster_labels[u] == cluster_labels[v] else 0
34         k_u = G.degree(u)
35         k_v = G.degree(v)
36         Q += (1 - (k_u * k_v) / (2 * m)) * delta
37     Q /= (2 * m)
```

```

38     return Q
39
40 Q = calculate_modularity(G, clusters)
41 Q = round(Q, 3)
42 print(f"Modularity Q: {Q}")
43
44 from collections import deque
45 import math
46
47 #####
48 ### ASSIGNMENT 2 ###
49 #####
50
51 # Define the graph edges
52 edges = [
53     (1, 2), (1, 3), (2, 3),
54     (3, 5), (3, 7), (4, 5),
55     (4, 6), (5, 6), (5, 7),
56     (7, 8), (7, 9), (8, 9)
57 ]
58
59 # Create the graph
60 G = nx.Graph()
61 G.add_edges_from(edges)
62
63 # SCAN algorithm parameters
64 epsilon = 0.8
65 mu = 2
66
67 # Step 1: Compute Structural Similarity between nodes
68 def structural_similarity(G, u, v):
69     neighbors_u = set(G.neighbors(u)) | {u}
70     neighbors_v = set(G.neighbors(v)) | {v}
71     intersection_size = len(neighbors_u & neighbors_v)
72     denom = math.sqrt(len(neighbors_u) * len(neighbors_v))
73     sim = intersection_size / denom if denom != 0 else 0
74     return round(sim, 3)
75
76 # Compute structural similarities for all connected node pairs
77 SS = {}
78 for u, v in G.edges():
79     sim = structural_similarity(G, u, v)
80     SS[(u, v)] = sim
81     SS[(v, u)] = sim # Since the graph is undirected
82
83 # Step 2: Identify Core Nodes and Clusters
84 nodes = list(G.nodes())
85 cluster_labels = {}
86 cluster_id = 0
87 visited = set()
88
89 def get_eps_neighbors(u):
90     return [v for v in G.neighbors(u) if SS.get((u, v), 0) >= epsilon]
91
92 for u in nodes:
93     if u in visited:
94         continue
95     N_eps = get_eps_neighbors(u)
96     if len(N_eps) >= mu - 1:
97         # Node u is a core node
98         cluster_id += 1
99         cluster_queue = deque()
100         cluster_queue.append(u)
101         cluster_nodes = set()
102         visited.add(u)
103         while cluster_queue:
104             current = cluster_queue.popleft()
105             cluster_nodes.add(current)
106             N_eps_current = get_eps_neighbors(current)
107             if len(N_eps_current) >= mu - 1:
108                 for w in N_eps_current:
109                     if w not in visited:
110                         visited.add(w)

```

```

111         cluster_queue.append(w)
112     for node in cluster_nodes:
113         cluster_labels[node] = f'Cluster {cluster_id}'
114     else:
115         visited.add(u)
116
117 # Step 3: Identify Hubs and Outliers
118 unclassified_nodes = set(G.nodes()) - set(cluster_labels.keys())
119 hubs = set()
120 outliers = set()
121
122 for u in unclassified_nodes:
123     neighbor_clusters = set()
124     for v in G.neighbors(u):
125         if v in cluster_labels:
126             neighbor_clusters.add(cluster_labels[v])
127     if len(neighbor_clusters) > 1:
128         hubs.add(u)
129     elif len(neighbor_clusters) == 0:
130         outliers.add(u)
131     else:
132         cluster_labels[u] = neighbor_clusters.pop()
133
134 # Output the results
135 print("\nClusters:")
136 clusters = {}
137 for node, label in cluster_labels.items():
138     clusters.setdefault(label, []).append(node)
139
140 for label, members in clusters.items():
141     print(f"{label}: {sorted(members)}")
142
143 print("\nHubs:")
144 print(sorted(hubs))
145
146 print("\nOutliers:")
147 print(sorted(outliers))

```